Code Analysis – Challenge Solution: Wonderland 3

By: Tehila Chananayev

Introduction

This reverse-engineering challenge presents an Assembly code that processes user input and validates it through a comparison mechanism. The primary goal is to understand the loop logic, determine the input requirements, and identify how the validation function operates to allow successful execution.

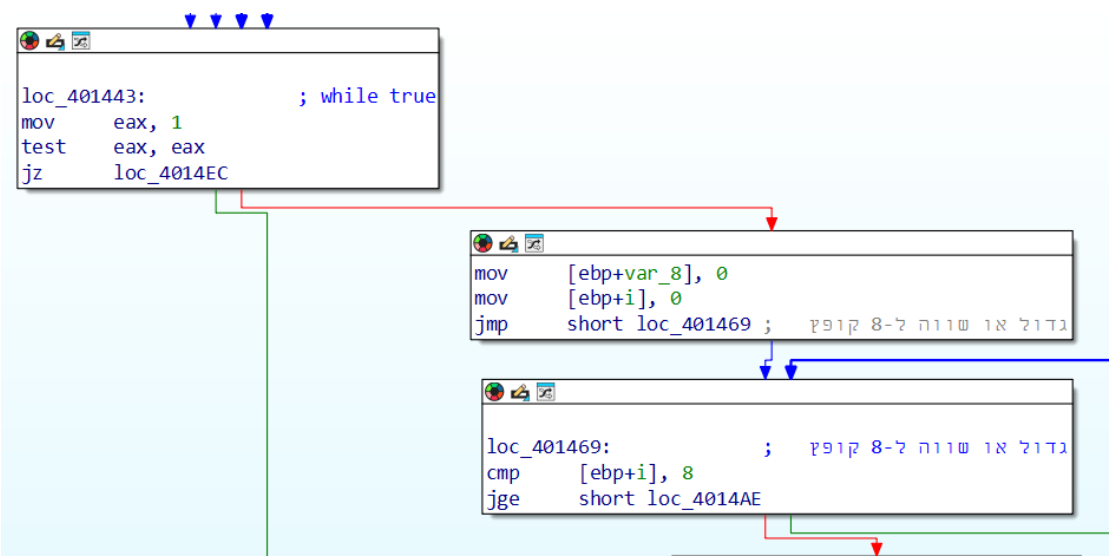
Code Structure

At the start of the program, there are initial print statements followed by the use of a while (true) loop, implemented with:

test eax, eax

Since eax is initialized to 1, the test always returns non-zero (ZF flag remains clear), resulting in an infinite loop. This is a typical way to represent while (true) in Assembly.

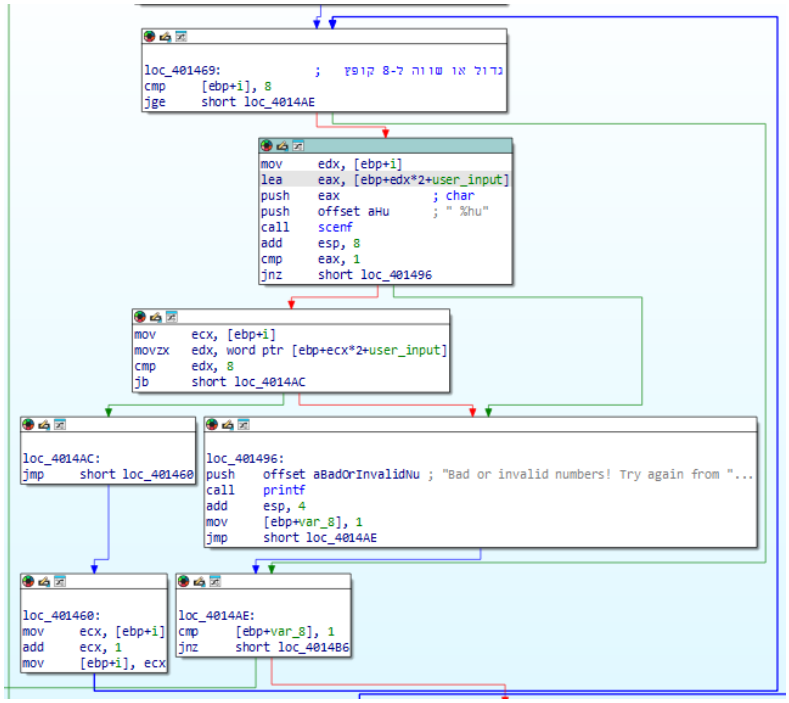
The loop processes input values while a counter i is less than 8. Initially, the variable was named var_4, but based on its behavior, we renamed it to i, as it increments by 1 in each iteration.



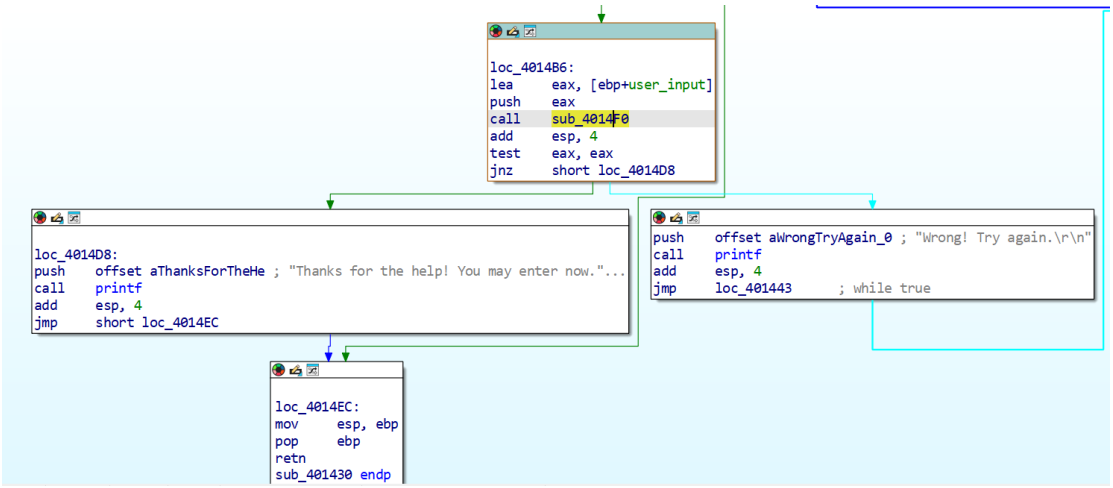
\

Core Logic

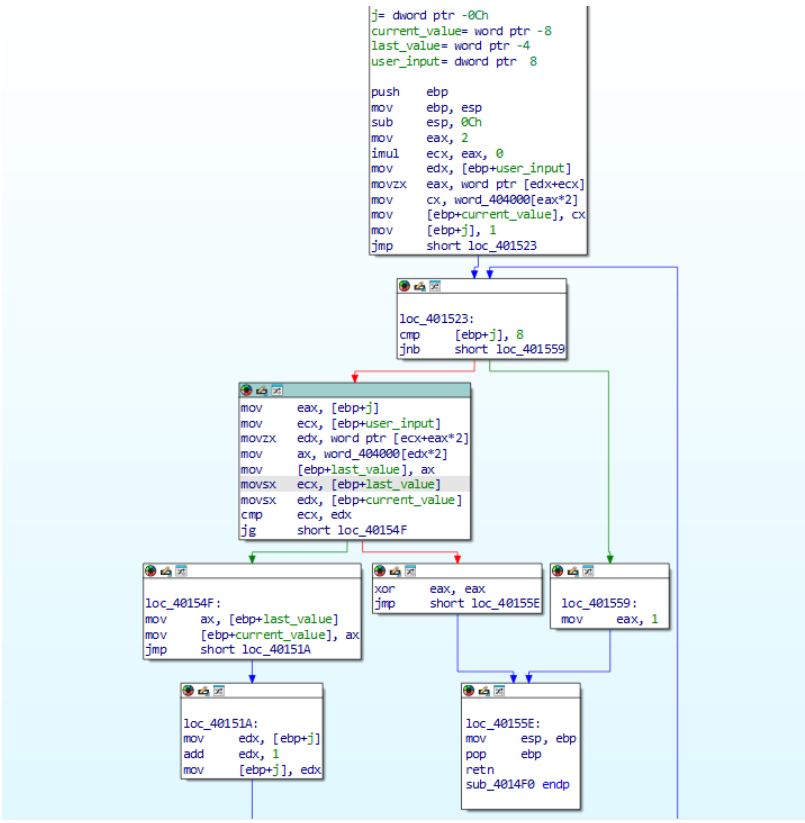
The loop uses scanf receives an address, enters it and advances by 2 bytes user_input each time we enter two bytes now we have an array of 8 unsigned short and if the input is greater than 8 we exit That is, we need to enter 8 numbers whose values are small, that is, values of 0-7 After collecting all 8 values, the program invokes a validation function.



That is, we realized that our entire right path is a path that receives 8 numbers from user_input and then it goes into our magic function, if it returns 1 it goes to success and if 0 then we failed.



Within this function:
1. The program initializes the validation by storing user_input[0] as the first value.
2. It enters a loop where each subsequent value is compared against the previous one.
3. If the current value is greater than the previous one, the loop continues.
4. If not, the function returns 0 (failure).
5. If all comparisons pass, the function returns 1 (success).



Key Variables

- user_input: An array of 8 values of type unsigned short, each value occupies 2 bytes.

- i: A loop counter incremented on each iteration.

- current_value (var_8): The current number being processed.

- last_value (var_4): The previous number to compare against.

Validation Function – Technical Flow

The validation function (shown in the image) operates as follows:
- Initializes the counter j = 1.
- Stores user_input[0] as the baseline (last_value).

- Loops while j < 8:
  - Loads user_input[j] into current_value.
  - Compares current_value > last_value.
  - If true, updates last_value and increments j.
  - If false, returns 0.
- If the loop completes, returns 1 (success).

Success Condition

To trigger the success path in the program, the user must input 8 strictly increasing numbers, all in the range of 0–7, such as:

0 1 2 3 4 5 6 7

These values are stored in memory using little endian format, meaning the least significant byte is stored first.

```
.data:00404000                  ;org 404000h
.data:00404000 word_404000      dw 7
.data:00404000
.data:00404002                  db  21h ; !
.data:00404003                  db    0
.data:00404004                  db    1
.data:00404005                  db    0
.data:00404006                  db 0A8h
.data:00404007                  db 0FDh
.data:00404008                  db  78h ; x
.data:00404009                  db 0ECh
.data:0040400A                  db 0F1h
.data:0040400B                  db    6
.data:0040400C                  db  0Dh
.data:0040400D                  db    0
.data:0040400E                  db  45h ; E
.data:0040400F                  db    0
.data:00404010 ; char *Str
```

```
Welcome to Wonderland. I am the mad hatter, and I have some riddles for you...
Input a level number (latest level- 3):
3


..Wait! Can you help me first with something?
(Enter the correct numbers)
4
3
2
0
6
1
7
5
Thanks for the help! You may enter now.
Welcome to Wonderland. I am the mad hatter, and I have some riddles for you...
Input a level number (latest level- 4):
```